

Reconstruction Metric - GPU Optimized

Karina

2026-02-17

Table of contents

0.1	1. Setup e Imports	1
0.2	2. Cargar Datos	2
0.3	3. Cargar SAE y Extraer Features	2
0.4	4. Split Train/Test y Filtrar BSPs de Piezas	3
0.5	5. Funciones GPU-Optimizadas	4
0.6	6. Fase 1: Identificar Features de Alta Precisión (GPU)	4
0.7	7. Fase 2: Reconstruir Tableros en Test Set (GPU)	7
0.8	8. Análisis de Resultados	10
0.9	9. Guardar Resultados	11

```
# Configuración para visualización en PDF
import pandas as pd
import numpy as np
import sys

pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 100)
np.set_printoptions(linewidth=100, edgeitems=3)

import os
os.environ['COLUMNS'] = '100'

print(" Configuración para PDF lista")
```

Configuración para PDF lista

0.1 1. Setup e Imports

```
import numpy as np
import torch
import sys
from pathlib import Path
from tqdm import tqdm
import time

project_root = Path('../..').resolve()
sys.path.insert(0, str(project_root))

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memoria: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
```

Device: cuda
GPU: NVIDIA GeForce RTX 5060
Memoria: 8.55 GB

0.2 2. Cargar Datos

```
# Cargar activaciones
activations_path = project_root / "sae" / "activations" / "data" / "layer5_200games.npy"
activations = np.load(activations_path)

print(f"Activaciones del modelo:")
print(f"  Shape: {activations.shape}")
print(f"  Memoria: {activations.nbytes / (1024**2):.2f} MB")
```

Activaciones del modelo:
 Shape: (11800, 512)
 Memoria: 23.05 MB

```
# Cargar ground truth de BSPs
bsp_gt_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.npy"
bsp_names_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.names.npy"

bsp_ground_truth = np.load(bsp_gt_path)
bsp_names = np.load(bsp_names_path, allow_pickle=True)

print(f"\nGround truth BSPs:")
print(f"  Shape: {bsp_ground_truth.shape}")
print(f"  Total BSPs: {len(bsp_names)}")
```

Ground truth BSPs:
 Shape: (11800, 198)
 Total BSPs: 198

0.3 3. Cargar SAE y Extraer Features

```
from sae.model.sae import SparseAutoencoder

input_dim = 512
hidden_dim = 16384

sae = SparseAutoencoder(input_dim, hidden_dim).to(device)
model_path = project_root / "sae" / "model" / "saved_model" / "sae_othello_best.pt"
checkpoint = torch.load(model_path, map_location=device)
sae.load_state_dict(checkpoint['model_state_dict'])
sae.eval()

print(f"SAE cargado:")
print(f"  Input: {input_dim}, Hidden: {hidden_dim}")
print(f"  Expansion: {hidden_dim/input_dim}x")
print(f"  Epoch: {checkpoint['epoch']}, Val MSE: {checkpoint['val_mse']:.6f}")
```

SAE cargado:
 Input: 512, Hidden: 16384
 Expansion: 32.0x
 Epoch: 29, Val MSE: 0.011182

```
# Extraer features del SAE en GPU
print("Extrayendo features del SAE...")
activations_tensor = torch.from_numpy(activations).float().to(device)

with torch.no_grad():
    sae_features = torch.relu(sae.encoder(activations_tensor))

print(f"\nFeatures SAE:")
print(f"  Shape: {sae_features.shape}")
print(f"  Device: {sae_features.device}")
print(f"  Sparsity: {(sae_features == 0).float().mean():.2%}")
print(f"  Activaciones promedio: {(sae_features > 0).sum(dim=1).float().mean():.1f}")
```

Extrayendo features del SAE...

```
Features SAE:
  Shape: torch.Size([11800, 16384])
  Device: cuda:0
  Sparsity: 96.54%
  Activaciones promedio: 567.5
```

0.4 4. Split Train/Test y Filtrar BSPs de Piezas

```
# Split: primeras 100 partidas = train, últimas 100 = test
n_moves = 59
split_idx = 100 * n_moves

sae_features_train = sae_features[:split_idx]
sae_features_test = sae_features[split_idx:]

bsp_gt_train = bsp_ground_truth[:split_idx]
bsp_gt_test = bsp_ground_truth[split_idx:]

print(f"Train set: {sae_features_train.shape}")
print(f"Test set: {sae_features_test.shape}")
```

```
Train set: torch.Size([5900, 16384])
Test set: torch.Size([5900, 16384])
```

```
# Filtrar BSPs de piezas (terminan en '1' o '2', no en '0')
bsp_pieces_indices = []
bsp_pieces_names = []

for i, name in enumerate(bsp_names):
    if len(name) == 6 and name.startswith('BSP') and not name.endswith('0'):
        bsp_pieces_indices.append(i)
        bsp_pieces_names.append(name)

bsp_pieces_indices = np.array(bsp_pieces_indices)

# Aplicar filtro y convertir a tensores GPU
bsp_gt_train_pieces = torch.from_numpy(bsp_gt_train[:, bsp_pieces_indices]).bool().to(device)
bsp_gt_test_pieces = torch.from_numpy(bsp_gt_test[:, bsp_pieces_indices]).bool().to(device)

print(f"\nBSPs para Reconstruction:")
print(f"  Total BSPs de piezas: {len(bsp_pieces_indices)}")
print(f"  Train shape: {bsp_gt_train_pieces.shape}")
print(f"  Test shape: {bsp_gt_test_pieces.shape}")
```

BSPs para Reconstruction:

Total BSPs de piezas: 128
Train shape: torch.Size([5900, 128])
Test shape: torch.Size([5900, 128])

0.5 5. Funciones GPU-Optimizadas

```
def fast_precision_score_gpu(y_true, y_pred, eps=1e-8):
    """
    Precisión vectorizada en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions, n_candidates) booleano

    Returns:
        precision_scores: Tensor (n_candidates,)
    """
    y_true_expanded = y_true.unsqueeze(1)

    tp = (y_true_expanded & y_pred).sum(dim=0).float()
    fp = (~y_true_expanded & y_pred).sum(dim=0).float()

    precision = tp / (tp + fp + eps)

    return precision

def fast_f1_score_gpu(y_true, y_pred, eps=1e-8):
    """
    F1 score vectorizado en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions,) booleano

    Returns:
        f1: Float
    """
    tp = (y_true & y_pred).sum().float()
    fp = (~y_true & y_pred).sum().float()
    fn = (y_true & ~y_pred).sum().float()

    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)

    f1 = 2 * (precision * recall) / (precision + recall + eps)

    return f1.item()

print(" Funciones GPU definidas")
```

Funciones GPU definidas

0.6 6. Fase 1: Identificar Features de Alta Precisión (GPU)

Para cada BSP, encontrar features con precisión 0.95 en el train set.

```

def identify_high_precision_features_gpu(
    sae_features_train,
    bsp_gt_train,
    precision_threshold=0.95,
    thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
    batch_size=512,
    verbose=True
):
    """
    Identifica features con precisión threshold para cada BSP (GPU OPTIMIZADO).

    Args:
        sae_features_train: Tensor (n_train, n_features) en GPU
        bsp_gt_train: Tensor (n_train, n_bsps) en GPU, bool
        precision_threshold: Mínima precisión requerida (default 0.95)
        thresholds: Umbrales de activación a probar
        batch_size: Features a procesar simultáneamente

    Returns:
        high_precision_features: dict {bsp_idx: [(feature_idx, threshold), ...]}
    """
    n_positions, n_features = sae_features_train.shape
    n_bsps = bsp_gt_train.shape[1]

    if verbose:
        print("="*60)
        print("FASE 1: IDENTIFICAR FEATURES DE ALTA PRECISIÓN")
        print("="*60)
        print(f"Threshold de precisión: {precision_threshold}")
        print(f"BSPs: {n_bsps}")
        print(f"Features: {n_features}")
        print(f"Batch size: {batch_size}")
        print("="*60)

    # Pre-calcular máximos
    f_max = sae_features_train.max(dim=0)[0]
    active_features = (f_max > 0).nonzero(as_tuple=True)[0]
    n_active = len(active_features)

    if verbose:
        print(f"\nFeatures activas: {n_active}/{n_features} ({n_active/n_features*100:.1f}%)")
        print()

    high_precision_features = {}
    thresholds_tensor = torch.tensor(thresholds, device=sae_features_train.device)

    pbar = tqdm(range(n_bsps), desc="Procesando BSPs") if verbose else range(n_bsps)

    for bsp_idx in pbar:
        bsp_labels = bsp_gt_train[:, bsp_idx]

        # Skip si nunca está activa
        if not bsp_labels.any():
            high_precision_features[bsp_idx] = []
            continue

        hp_features_for_bsp = []
        features_found = torch.zeros(n_active, dtype=torch.bool, device=sae_features_train.device)

        # Procesar features activas en batches
        for batch_start in range(0, n_active, batch_size):
            batch_end = min(batch_start + batch_size, n_active)

```

```

        batch_features_idx = active_features[batch_start:batch_end]

        # Extraer features del batch
        batch_activations = sae_features_train[:, batch_features_idx]
        batch_f_max = f_max[batch_features_idx]

        # Probar cada threshold
        for t in thresholds_tensor:
            # Binarizar: (n_positions, batch_size)
            predictions = batch_activations > (t * batch_f_max.unsqueeze(0))

            # Solo calcular precisión si hay predicciones
            has_predictions = predictions.any(dim=0)

            if has_predictions.any():
                # Calcular precisión solo para features con predicciones
                precision_scores = torch.zeros(predictions.shape[1], device=sae_features_train.device)
                precision_scores[has_predictions] = fast_precision_score_gpu(
                    bsp_labels,
                    predictions[:, has_predictions]
                )

                # Encontrar features que cumplen threshold
                good_features = precision_scores >= precision_threshold

                if good_features.any():
                    # Guardar features encontradas
                    local_indices = good_features.nonzero(as_tuple=True)[0]
                    for local_idx in local_indices:
                        global_idx = batch_start + local_idx.item()
                        if not features_found[global_idx]:
                            feature_idx = batch_features_idx[local_idx].item()
                            hp_features_for_bsp.append((feature_idx, t.item()))
                            features_found[global_idx] = True

            high_precision_features[bsp_idx] = hp_features_for_bsp

        return high_precision_features

print(" Función Fase 1 definida")

```

Función Fase 1 definida

```

# Ejecutar Fase 1
start_time = time.time()

high_precision_features = identify_high_precision_features_gpu(
    sae_features_train,
    bsp_gt_train_pieces,
    precision_threshold=0.95,
    batch_size=512,
    verbose=True
)

phase1_time = time.time() - start_time

# Estadísticas
n_features_per_bsp = [len(features) for features in high_precision_features.values()]

print("\n" + "="*60)
print("RESULTADOS FASE 1")
print("="*60)

```

```

print(f"Total BSPs: {len(high_precision_features)}")
print(f"Features promedio por BSP: {np.mean(n_features_per_bsp):.1f}")
print(f"BSPs con 0 features: {np.sum(np.array(n_features_per_bsp) == 0)}")
print(f"BSPs con 1+ features: {np.sum(np.array(n_features_per_bsp) > 0)}")
print(f"BSPs con 5+ features: {np.sum(np.array(n_features_per_bsp) >= 5)}")
print(f"\nTiempo Fase 1: {phase1_time:.2f} seg")
print("="*60)

```

FASE 1: IDENTIFICAR FEATURES DE ALTA PRECISIÓN

Threshold de precisión: 0.95

BSPs: 128

Features: 16384

Batch size: 512

Features activas: 7775/16384 (47.5%)

Procesando BSPs: 100% | 128/128 [02:48<00:00, 1.32s/it]

RESULTADOS FASE 1

Total BSPs: 128

Features promedio por BSP: 1889.9

BSPs con 0 features: 0

BSPs con 1+ features: 128

BSPs con 5+ features: 128

Tiempo Fase 1: 168.66 seg

0.7 7. Fase 2: Reconstruir Tableros en Test Set (GPU)

```

def reconstruct_boards_gpu(
    sae_features_test,
    bsp_gt_test,
    high_precision_features,
    f_max,
    batch_size=1000,
    verbose=True
):
    """
    Reconstruye tableros usando features de alta precisión (GPU COMPLETAMENTE VECTORIZADO).

    Args:
        sae_features_test: Tensor (n_test, n_features) en GPU
        bsp_gt_test: Tensor (n_test, n_bsps) en GPU, bool
        high_precision_features: dict {bsp_idx: [(feature_idx, threshold), ...]}
        f_max: Tensor (n_features,) - máximos calculados del train set
        batch_size: Posiciones a procesar simultáneamente

    Returns:
        reconstruction_score: F1 promedio
        f1_per_position: Array de F1 por posición
    """

```

```

"""
n_positions, n_features = sae_features_test.shape
n_bsps = bsp_gt_test.shape[1]
device = sae_features_test.device

if verbose:
    print("="*60)
    print("FASE 2: RECONSTRUIR TABLEROS (GPU VECTORIZADO)")
    print("="*60)
    print(f"Posiciones: {n_positions}")
    print(f"BSPs: {n_bsps}")
    print(f"Batch size: {batch_size}")
    print("="*60)
    print()

# PRE-PROCESAR: Convertir high_precision_features a tensores GPU
if verbose:
    print("Pre-procesando features de alta confianza...")

# Para cada BSP, crear matriz de (n_features, max_features_per_bsp)
# Usaremos -1 para padding
max_hp_features = max(len(feats) for feats in high_precision_features.values())

# Matrices: (n_bsps, max_hp_features)
hp_feature_indices = torch.full((n_bsps, max_hp_features), -1, dtype=torch.long, device=device)
hp_thresholds = torch.zeros((n_bsps, max_hp_features), device=device)

for bsp_idx, hp_features in high_precision_features.items():
    for i, (feat_idx, threshold) in enumerate(hp_features):
        hp_feature_indices[bsp_idx, i] = feat_idx
        hp_thresholds[bsp_idx, i] = threshold

if verbose:
    print(f" Pre-procesamiento completo")
    print(f" Max features por BSP: {max_hp_features}")
    print()

# Procesar en batches
fl_scores = []
n_batches = (n_positions + batch_size - 1) // batch_size

pbar = tqdm(range(n_batches), desc="Procesando batches") if verbose else range(n_batches)

for batch_idx in pbar:
    start_idx = batch_idx * batch_size
    end_idx = min(start_idx + batch_size, n_positions)
    batch_n_positions = end_idx - start_idx

    # Features del batch: (batch_n_positions, n_features)
    batch_features = sae_features_test[start_idx:end_idx]
    batch_gt = bsp_gt_test[start_idx:end_idx]

    # Inicializar predicciones: (batch_n_positions, n_bsps)
    predictions = torch.zeros((batch_n_positions, n_bsps), dtype=torch.bool, device=device)

    # Para cada BSP, vectorizar la detección
    for bsp_idx in range(n_bsps):
        # Features de alta confianza para esta BSP
        feat_indices = hp_feature_indices[bsp_idx] # (max_hp_features,)
        thresholds = hp_thresholds[bsp_idx] # (max_hp_features,)

        # Filtrar padding (-1)

```



```

        valid_mask = feat_indices >= 0
        if not valid_mask.any():
            continue

        feat_indices = feat_indices[valid_mask]
        thresholds = thresholds[valid_mask]

        # Extraer activaciones de estas features: (batch_n_positions, n_valid_features)
        selected_features = batch_features[:, feat_indices]

        # Calcular thresholds escalados: (n_valid_features,)
        scaled_thresholds = thresholds * f_max[feat_indices]

        # Verificar si alguna feature supera su threshold: (batch_n_positions, n_valid_features)
        activated = selected_features > scaled_thresholds.unsqueeze(0)

        # Si alguna feature se activa, predecir True: (batch_n_positions,)
        predictions[:, bsp_idx] = activated.any(dim=1)

    # Calcular F1 para cada posición del batch
    for i in range(batch_n_positions):
        pred = predictions[i]
        gt = batch_gt[i]

        if pred.any() or gt.any():
            f1 = fast_f1_score_gpu(gt, pred)
        else:
            f1 = 1.0

        f1_scores.append(f1)

    reconstruction_score = np.mean(f1_scores)

    return reconstruction_score, f1_scores

print(" Función Fase 2 definida")

```

Función Fase 2 definida

```

# Ejecutar Fase 2
start_time = time.time()

# Usar f_max del train set
f_max_train = sae_features_train.max(dim=0)[0]

reconstruction_score, f1_per_position = reconstruct_boards_gpu(
    sae_features_test,
    bsp_gt_test_pieces,
    high_precision_features,
    f_max_train,
    batch_size=1000, # Procesar 1000 posiciones a la vez
    verbose=True
)

phase2_time = time.time() - start_time
total_time = phase1_time + phase2_time

print("\n" + "="*60)
print("RESULTADO RECONSTRUCTION")
print("="*60)
print(f"Reconstruction Score: {reconstruction_score:.4f}")
print(f"Objetivo (paper): 0.95")

```

```

print(f"Diferencia: {reconstruction_score - 0.95:+.4f}")
print(f"\nTiempo Fase 2: {phase2_time:.2f} seg")
print(f"Tiempo Total: {total_time:.2f} seg ({total_time/60:.2f} min)")
print("="*60)

```

```

=====
FASE 2: RECONSTRUIR TABLEROS (GPU VECTORIZADO)
=====

```

```

Posiciones: 5900
BSPs: 128
Batch size: 1000
=====

```

Pre-procesando features de alta confianza...

```

Pre-procesamiento completo
Max features por BSP: 4618

```

```

Procesando batches: 100%|      | 6/6 [00:04<00:00, 1.39it/s]

```

```

=====
RESULTADO RECONSTRUCTION
=====

```

```

Reconstruction Score: 0.2292
Objetivo (paper): 0.95
Diferencia: -0.7208

```

```

Tiempo Fase 2: 21.40 seg
Tiempo Total: 190.06 seg (3.17 min)
=====

```

0.8 8. Análisis de Resultados

```

# Distribución de F1 scores
f1_array = np.array(f1_per_position)

print("Distribución de F1 por posición:")
print(f"  Mínimo: {np.min(f1_array):.4f}")
print(f"  Máximo: {np.max(f1_array):.4f}")
print(f"  Media: {np.mean(f1_array):.4f}")
print(f"  Mediana: {np.median(f1_array):.4f}")
print(f"  Std: {np.std(f1_array):.4f}")
print()
print(f"Posiciones con F1 > 0.95: {np.sum(f1_array > 0.95)} ({np.mean(f1_array > 0.95):.1%})")
print(f"Posiciones con F1 > 0.90: {np.sum(f1_array > 0.90)} ({np.mean(f1_array > 0.90):.1%})")
print(f"Posiciones con F1 < 0.80: {np.sum(f1_array < 0.80)} ({np.mean(f1_array < 0.80):.1%})")

```

Distribución de F1 por posición:

```

Mínimo: 0.0000
Máximo: 1.0000
Media: 0.2292
Mediana: 0.1176
Std: 0.2549

```

```

Posiciones con F1 > 0.95: 285 (4.8%)
Posiciones con F1 > 0.90: 307 (5.2%)
Posiciones con F1 < 0.80: 5546 (94.0%)

```

```
# Top 10 posiciones mejor reconstruidas
top_indices = np.argsort(f1_array)[-10:][::-1]

print("\nTop 10 mejor reconstruidas:")
print("="*60)
print(f"{'Partida':<10} {'Movimiento':<12} {'F1':<8}")
print("-"*60)
for idx in top_indices:
    game_idx = idx // 59
    move_idx = idx % 59
    print(f"{game_idx:<10} {move_idx:<12} {f1_per_position[idx]:.4f}")
```

Top 10 mejor reconstruidas:

```
=====
Partida      Movimiento      F1
-----
```

0	2	1.0000
0	1	1.0000
89	0	1.0000
89	1	1.0000
14	2	1.0000
14	1	1.0000
14	0	1.0000
88	2	1.0000
88	1	1.0000
88	0	1.0000

```
# Bottom 10 posiciones peor reconstruidas
bottom_indices = np.argsort(f1_array)[:10]

print("\nBottom 10 peor reconstruidas:")
print("="*60)
print(f"{'Partida':<10} {'Movimiento':<12} {'F1':<8}")
print("-"*60)
for idx in bottom_indices:
    game_idx = idx // 59
    move_idx = idx % 59
    print(f"{game_idx:<10} {move_idx:<12} {f1_per_position[idx]:.4f}")
```

Bottom 10 peor reconstruidas:

```
=====
Partida      Movimiento      F1
-----
```

83	49	0.0000
46	15	0.0000
46	17	0.0000
46	24	0.0000
46	39	0.0000
46	47	0.0000
15	53	0.0000
71	58	0.0000
19	58	0.0000
84	12	0.0000

0.9 9. Guardar Resultados

```

# Guardar resultados
results = {
    'reconstruction_score': reconstruction_score,
    'f1_per_position': f1_per_position,
    'phase1_time_seconds': phase1_time,
    'phase2_time_seconds': phase2_time,
    'total_time_seconds': total_time,
    'n_features_per_bsp': n_features_per_bsp,
    'bsp_names': bsp_pieces_names
}

output_path = project_root / "sae" / "metrics" / "04_results" / "reconstruction_results.npz"
np.savez(output_path, **results)

print(f" Resultados guardados en:")
print(f"  {output_path.name}")

```

Resultados guardados en:
 reconstruction_results.npz